PROJECT REPORT II

Technical Implementation and Test Validation

Linux Debugging System with Memory Leak Detection and Reporting Tool

KL University | CSE | Hackathon: 17 October 2026

Guided by	Dr. K. Anusha
Team members	2520030034 - P. Prathik 2520030188 - U. Praneeth 2520030573 - M. Harshith
Report focus	Implementation details, command-line operation, and observed test evidence

This document is a technical companion to the project overview. It concentrates on how the prototype behaves in implementation and testing, rather than repeating the first report's general overview.

1. Scope of This Technical Report

This report records the internal tracking approach, the commands used to run the prototype, and the results observed during the Ubuntu demonstration. It is intended to complement the project overview report by providing concrete implementation and validation detail.

2. Implementation Components

File / component	Responsibility	How it is used
src/main.c	Test harness and command-line entry point	Runs leak, clean, partial-release, repeated-release-safety, and help scenarios.
src/leak_tracker.c	Allocation registry and report logic	Records active blocks, updates releases, totals bytes, and writes report output.
src/leak_tracker.h	Tracker API and macros	Provides TRACK_MALLOC, TRACK_CALLOC, and TRACK_FREE wrappers.
Makefile	Build and test automation	Compiles the executable and runs the automated test script.
tests/run_tests.sh	Automated checks	Checks output counts, statuses, report creation, and invalid-mode behavior.
reports/memory_report.txt	Human-readable result file	Stores the latest report written by the selected demonstration run.

3. Tracking Logic and Data Lifecycle

The tracker maintains a registry of allocations that passed through its own wrapper functions. A record retains the pointer, requested size, allocation source file and line, and whether the block is still active. The reporting function scans active records at report time.

1. Allocate TRACK_MALLOC	2. Record pointer + size	3. Release TRACK_FREE	4. Scan active records	5. Print + save report

Interpretation: if a tracked record is still active when the report is generated, the tool lists it as a potential leak. The word "potential" is important because a program may deliberately retain memory until exit.

4. Running the Prototype in Ubuntu

From the extracted project folder, run the following commands one at a time:

```
cd ~/hackathon/Linux-Debugging-System
make
./build/linux_debugging_system
```

The interactive menu executes the selected test and then returns to the Ubuntu shell. Start the program again for each additional menu test. Direct modes are also supported: `leak`, `clean`, `partial`, `invalid-free`, and `--help`.

5. Observed Validation Results

The following results are based on the terminal output produced during the demonstration. These are observed outcomes, not merely planned values.

Scenario	Observed output	Status	What it confirms
Intentional leak	1 outstanding block; 100 bytes; source shown as src/main.c:20	PASS	The unreleased tracked block is reported.
Correct release	1 allocation; 1 released block; 0 outstanding blocks; 0 bytes	PASS	A tracked release removes the block from the outstanding set.
Partial release	3 allocations; 2 released; 1 block of 200 bytes remains	PASS	Only the block not released is reported.
Repeated-release safety	0 outstanding blocks; 0 bytes; second call described as a no-op after pointer becomes NULL	PASS for this test	The demonstration avoids calling free twice on the same non-NULL pointer.
Help option	Usage and supported mode names displayed	PASS	The user can discover available commands.

6. Report File and Evidence

The default report path is `reports/memory_report.txt`. It includes totals for allocations and releases, outstanding block details, byte totals, status, and the tracked allocation location where available. Because each new run writes to the default path, the report file represents the most recent test.

```
cat reports/memory_report.txt
```

For an evaluation screenshot that shows a detected leak, run `./build/linux_debugging_system leak` immediately before displaying the report.

7. What the Current Prototype Does Not Claim

- It tracks allocations made through TRACK_MALLOC / TRACK_CALLOC and releases routed through TRACK_FREE; direct untracked calls are outside its visibility.
- It reports outstanding tracked allocations as potential leaks; it does not infer every program's intended memory lifetime.
- The repeated-release demonstration is safe because the pointer is set to NULL after release. This does not constitute comprehensive detection of all invalid-free bugs.
- It is a teaching and hackathon prototype for controlled local test programs, not a kernel debugger or a scanner for arbitrary running processes.

8. Next Technical Improvements

- Add tests for allocation failure and tracking-table capacity limits.
- Improve diagnostics for pointers not present in the registry.
- Offer a distinct report file for each run so earlier evidence is preserved.
- Integrate the detector command into the team's broader Linux Debugging System menu.
- Compare test programs against Valgrind or AddressSanitizer during development.

9. Conclusion

The executed tests demonstrate the central behavior of the prototype: tracked allocation records are created, releases update those records, and remaining active records are summarized in a report. The actual Ubuntu results show a 100-byte intentional leak, a clean-release case with zero outstanding bytes, and a partial-release case with one remaining 200-byte block. The implementation scope remains explicit so the team can explain both the evidence and the limitations accurately.